Table of Contents

(*Proposal draft begins on page 6*)

Table of Contents **1**

Initial Project Mock Up: **2**

Updated Project Overview: **4**

Checklist: **5**

Proposal Draft (Typical requirements from the capstone project website) **6**

2. Objectives 6

3. Background 6

4. What will be done 7

5. How does ChalkStream relate to the degree program of each student? 7

6. A discussion of how the group, collectively, has the skills required to undertake the project. 7

7. Method(s) you are going to use in solving the problem and how they relate to knowledge you acquired in your degree program to-date. 7

8. A proposed timetable for completion of the project including major intermediate milestones. 7

9. Project risks and mitigation strategies. 7

10. Required special components and facilities that you require. 8

11. References 8

Initial Project Mock Up:

1. ChalkStream

ChalkStream is an AI-powered classroom assistant that digitizes traditional blackboards and whiteboards in real time. The system utilizes computer vision and machine learning to capture writing and diagrams from a physical board and transform them into structured, editable digital content.

Students can view the content in real time using the project's app/website, allowing them to access the content with a wide variety of functions:

- Note-taking: The project aims to create a base of study notes for the student that can be personalized with minimal effort, allowing students to put more effort into engaging with the lecture rather than spending time creating study notes. The students may highlight, make adjustments, or discard the notes, as well as organize the notes.
- Mid-lecture recap: The project allows students to view a recap of the lecture, allowing students who are falling behind to quickly catch up mid-lecture.
- Question answering: The project integrates ChatGPT, allowing students to directly use AI to explain selected topics. The project also has a help button. The professor can be notified if a sizable portion of the students are struggling with the topic.
- Lecture translation: For students who have English as a second language, this project aims to provide a translation of the content to these students. Allowing them to understand the lecture more easily.
- Organization: Student's study notes can be saved within the application/website, which will be organized by courses and by date within each course.

This project addresses the gap between traditional teaching methods and modern digital learning platforms by enabling instructors to continue using physical boards while retaining the ability to provide up-to-date, remote-friendly, accessible, and searchable lecture materials. Additionally, the project provides tools to create personalized assistance for students, allowing them to keep up to fast-paced lectures.

The project addresses commonly faced problems by students in lectures, especially with challenging courses where the material content is long and complex and is limited by time constraints. One of the key features of this project is extracting main concepts and important topics from lecture content and using it to create resources that are coordinated, specific and useful to students aiding their learning and supporting their studies. It also addresses language barriers, difficulty adapting to different teaching styles, extracting useful content from complex and lengthy lectures. Staying focused and attentive for long periods of time is a constant challenge faced by learners. ChalkStream aims to solve these problems.

Key Feature:

- Real-Time Capture & Rendering: Camera feed is processed to detect strokes, text, diagrams, and equations. These are redrawn digitally as vector graphics on a live canvas.
- Handwriting & Math Recognition: Convert handwritten notes into editable text and mathematical expressions into LaTeX for perfect digital formatting.
- Lecture Synchronization: Record and synchronize audio/video with board content, enabling searchable lecture playback.
- Streaming & Remote Access: Broadcast a digital board in real-time for remote students.
- Post-Lecture Editing & Export: Allow resizing, correction, and reorganization of notes and diagrams. Export to PDF, LaTeX, or interactive formats.

- AI-Powered Enhancements?: Automatic summarization, keyword tagging, and quiz generation based on lecture content.
- Live AI help: Students will be provided an app that allows them to review the board notes in Realtime. The note is interactive such that each note can be personally highlighted and discarded, and an AI assistant (connected to ChatGPT) will be provided to further explain each concept if necessary.
    - professors can provide the general topic beforehand, such that the AI could provide a more accurate answer for the students.
    - Students can translate the notes into their preferred language if english is their second language
    - Students can get summaries based on their learning style (Auditory, Visual, etc …)

How is ChalkStream different from existing product lines like at Mimio?

While products like Mimio focus on creating a more interactive environment for students to engage with lecture content and improve participation, we aim to empower the student directly with intuitive, AI-assisted tools for personalized learning. Rather than just digitizing the instructor's content, our system transforms audio and visual recordings into structured, searchable, and adaptable study material. Students can highlight, annotate, translate, and receive instant AI-driven explanations or summaries based on their preferred learning style. In doing so, we shift the emphasis from general classroom participation to individual understanding, retention, and accessibility.

With Mimio content is digitized, but no deep accessibility features (translations, multi-format learning, transcripts, live search). ChalkStream directly bridges this gap. By leveraging AI, it transforms raw lecture notes into structured, searchable, and personalized study material. Students can highlight, reorganize, and interact with their notes while receiving instant AI-generated explanations or summaries tailored to their preferred learning style (visual, auditory, or textual). Additionally, built-in translation features support ESL learners, ensuring that students from diverse backgrounds can access and understand lecture content more effectively. In this way, ChalkStream shifts the focus from simply digitizing instructor material to empowering individual students with adaptive, inclusive, and intelligent learning tools.

How is ChalkStream different from existing board recognition product lines like Logitech Scribe?

While Logitech Scribe is designed as a corporate meeting tool to make whiteboard content visible to remote participants, ChalkStream goes far beyond simple capture and cleanup. Scribe provides a polished video feed but leaves the content static and passive. ChalkStream, by contrast, transforms the lecture board into structured, interactive study material that students can search, reorganize, and personalize in real time. Instead of AI being used only for visual enhancement, ChalkStream leverages it for adaptive learning: generating summaries tailored to different learning styles, translating notes for ESL students, and offering live explanations or recaps when concepts are unclear. Moreover, ChalkStream provides instructors with real-time insight into where students are struggling, enabling responsive teaching. In short, Scribe digitizes the board for visibility; ChalkStream digitizes and enriches it for comprehension, accessibility, and personalized learning — making it a fundamentally better fit for education.

Updated Project Overview:

ChalkStream: Intelligent Whiteboard Capture and Engagement Platform

ChalkStream is a hybrid learning platform designed to turn the classroom lecture into a comprehensive, digital-first experience. It captures a physical whiteboard as an editable digital structure, spoken explanations through transcripts, hosts interactive polls, takes attendance, and uses AI to integrate this information into a single, browser-based environment where students can learn, engage, and review in our one-stop shop application.

The system is built on three domains: real-time visual capture, AI-based transcription, and integrated engagement.

1. **Real-time Visual Capture**

- A standard USB or laptop camera records the whiteboard.
- Computer vision algorithms:
    - Detect and crop the board surface.
    - Correct perspective distortions and reduce glare.
    - Apply presenter transparency so writing remains visible even when blocked.

The result is a clean, digitally enhanced whiteboard stream optimized for both in-room projection and remote learning.

2. **AI-Based Transcription and Structuring**

- ChalkStream's AI doesn't stop at making the board visible, it makes it usable:
- Handwriting Recognition: Converts equations, diagrams, and notes into editable formats such as LaTeX, Markdown, or vector graphics.
- Contextual Structuring: Groups notes into logical sections (e.g., definitions, examples, proofs) by analyzing layout and lecture flow.
- Summarization: Produces concise lecture highlights and learning objectives at the end of class.
- Archiving: Stores data in custom access layers, so professors can limit what content is saved publicly or privately. While still enabling AI insights on student questions

3. **Integrated Engagement Layer**

ChalkStream embeds engagement features that are normally fragmented across different apps:

- Attendance Tracking through QR codes or device sign-ins.
- Knowledge Checks are tied directly to lecture content.
- Live Annotations that let students comment or highlight on synchronized board sections.

By orchestrating these within the same browser session, ChalkStream eliminates the friction of juggling Zoom, Wooclap, Google Docs, and screenshots separately.

Solving a Concrete Gap:

Competing tools address only slices of the problem. Zoom and Google Meet provide raw video feeds but no transcription, structuring, or searchable archives. ShareTheBoard improves visibility and captures snapshots, but it lacks any organization of content into learning-ready resources, native audio recording and syncing, or integration with polls and attendance. ChalkStream's innovation is to treat the lecture as multiple streams of data, visuals, texts, and interactions, and orchestrate it with AI so students

receive a single, browser-based learning hub. By anchoring outputs to captured lecture content, this approach compensates for the probabilistic behavior of AI systems, which can sometimes yield inaccuracies or gaps in responses.

Engineering Design:

The project requires building a low-latency pipeline that combines camera input, computer vision, and multiple AI services into a synchronized feed. Key design challenges include evaluating handwriting recognition APIs versus open-source models, structuring data into knowledge graphs for searchable archives, and ensuring that engagement features (like polls and annotations) remain synchronized with the evolving board. These design problems require trade-off analysis, prototyping, and orchestration of several systems.

Summary:

ChalkStream is more than a whiteboard camera. It is an engineered system that acquires as much relevant data as possible during a lecture, orchestrates it with AI, and delivers it through a single, integrated browser experience. The result is a platform where physical teaching and digital learning converge, making lectures more accessible, interactive, and valuable long after class ends.

Proposal draft due date: 8:00pm Monday, September 29th, meeting

Checklist:

1. Proposal draft
2. System block diagram (with explanations detailing inputs, outputs, and alternative implementations)
3. Proposed timeline for development of each component of the system with Gantt chart view
4. Storage location for capstone work products

Proposal due date: Friday Oct 17th

Proposal Draft (Typical requirements from the capstone project website)

1. Title Page

ChalkStream #62

Cameron Bodkin-Dick #101-272-935
Sandy Alzabadani #101-055-734
Eric Cui #101-237-617
Safaa Bedawi (& Adrian Chan)

2. Objectives

The objective of ChalkStream is to design and implement a real-time classroom assistant that bridges the gap between traditional board-based instruction and modern digital learning environments. Our motivation is rooted in a recurring challenge, where students struggle to balance the tasks of transcribing lecture material while engaging meaningfully with the content. It is impractical for students to split their focus between these tasks. By engineering a system that automatically captures, processes, enriches, and organizes lecture content in digital form, ChalkStream aims to enhance comprehension, accessibility, and long-term study outcomes.

This project takes an engineering approach by breaking down the lecture pipeline into measurable subsystems: vision capture, recognition, synchronization, AI-based enrichment, and delivery. Each subsystem is tied to functional requirements that will be validated using quantitative benchmarks, as well as non-functional requirements that guarantee robustness, usability, and scalability in real-world classrooms. Collectively, these objectives define ChalkStream as not only a demonstration of technical feasibility but also a tool that meaningfully impacts the student learning experience.

Functional Requirements:

1. Real-Time Capture:
    a. The system shall acquire a continuous video feed from a webcam or connected camera.
    b. The system shall automatically detect and isolate the whiteboard or chalkboard region, applying perspective correction to produce a flat, readable image.
    c. The system shall provide instructor-level configuration for board type (whiteboard or chalkboard), enabling tailored enhancement protocols (brightness, contrast, sharpening, grayscale normalization).
2. Content Segmentation:
    a. The system shall segment captured board content into categories: text, mathematical expressions, and diagrams.
    b. Each segment shall be routed to specialized processing pipelines (OCR, equation recognition, or diagram reconstruction)
3. Text and Handwriting OCR (Optical Character Recognition):
    a. The system shall process handwritten text from the board into structured, searchable digital text.
    b. Recognized text shall be stored for lecture digitization, content indexing, and AI-assisted insights.
4. Mathematical Expression Recognition:
    a. The system shall recognize handwritten equations and convert them into LaTeX format.
    b. Converted expressions shall be linked to the lecture timeline for digitization, search, and enrichment tasks.
5. Diagram Recognition & Reconstruction:

    a. The system shall convert hand-drawn diagrams into clean, vectorized representations.
    b. Vectorized diagrams shall be stored and integrated into lecture playback for accurate reproduction.
6. Speech Transcription:
    a. The system shall capture instructor audio via a microphone.
    b. The system shall produce accurate transcripts, time-aligned with board content, for both live delivery and playback search.
7. Synchronization and Search:
    a. The system shall align text, equations, diagrams, and transcripts into a unified timeline.
    b. The system shall support keyword-based search with playback navigation to the relevant lecture segment.
8. AI-Based Enrichment:
    a. The system shall generate contextual summaries of lecture segments.
    b. The system shall provide language translation of captured text and transcripts.
9. Instructor Experience:
    a. The system shall allow instructors to control lecture visibility and student access (e.g., real-time, post-lecture, or restricted).
    b. The system shall enable quiz generation and facilitation from lecture content or instructor-supplied material.
    c. The system shall track student attendance and engagement, including time spent in the lecture session.
10. Student Interaction:
    a. The system shall allow students to create personalized notes by highlighting, annotating, and organizing lecture content.
    b. The system shall support exporting personalized lecture notes in common formats (e.g., PDF, LaTeX).

Non-functional requirements:

1. Performance: Maintain processing latency below 500 milliseconds per component, maintaining smooth performance during live use.
2. Scalability: Support simultaneous access by at least 50 concurrent students without service degradation.
3. Modularity: Design system components with replaceable recognition engines, allowing iterative improvement.
4. Reliability: Maintain 95% uptime during scheduled use, with self-recovery mechanisms in case of component failures.
5. Security and Privacy: Apply encryption in transit and at rest, with instructor-level controls for archiving, deletion, and sharing of captured data after the lecture ends.
6. Usability: Provide an accessible browser-based interface, so students and instructors can use the system intuitively without specialized training.

These objectives establish ChalkStream as both an applied research project and a practical engineering solution. Success will be measured through carefully chosen metrics, recognition accuracy, latency benchmarks, transcript error rates, and usability studies. In doing so, ChalkStream will not only demonstrate technical innovation but also show clear educational value, directly addressing the challenges faced in fast-paced, content-heavy lectures.

3. Background

Traditional lecture delivery relies heavily on handwritten notes from whiteboards and chalkboards. While it's a common strategy for real-time explanation, this teaching method still creates significant barriers for students who struggle to keep up with note-taking, particularly in technical or mathematically intensive courses. Additionally, it brings complexities for students who require asynchronous accommodations. The result is that students often miss key content or spend more time transcribing than understanding. Numerous digital solutions attempt to bridge this gap, but they fall short in specific ways.

Video conferencing platforms such as Zoom or Google Meet provide live video feeds and automatic transcription of speech. However, the board content remains a static video stream, unsearchable and not engaging for the student. Interactive classroom tools such as Wooclap or Mentimeter enable polls and engagement but do not integrate with lecture content. Hardware-driven products such as Mimio digitize board strokes but primarily focus on instructor presentation rather than empowering student comprehension. Similarly, Logitech Scribe improves board visibility for remote participants but leaves content as a passive video feed, with no structuring, searchability, or personalization. ShareTheBoard, another competitor, enhances legibility but does not restructure content into editable notes or align it with lecture transcripts.

The state of the art in this field emphasizes visibility and participation but rarely comprehension and organization. What sets ChalkStream apart is its deliberate focus on transforming board content into structured, usable study material. Instead of just seeing the board more clearly, students will have access to searchable notes, equation transcriptions, diagram reconstructions, synchronized transcripts, and AI-based enhancements. This elevates ChalkStream beyond digitization into the realm of adaptive learning.

4. What will we do?

The ChalkStream system is constructed as a pipeline of components. Each component is described here in terms of its purpose, inputs, outputs, and various implementation methods.

4.1.1 Image Capture

Description: Video feed from a camera in the lecture hall, pointed as uniformly and clearly as possible/convenient at the board.
Outputs: Video feed of physical board
Implementations:

- Integrated Webcam
    - The native camera on a laptop device (commonly 1080p resolution).
- USB Connected Camera
    - A USB camera, traditionally on a tripod that can be separate from the device and positioned towards the board (commonly higher resolution 1440p+)
- Overhead lecture camera
    - A permanent camera affixed to the top of some lecture halls with Bluetooth or USB connection for instructors

4.1. Frame Preprocessing
Description: Detect the board area, correct perspective, and keep writing visible even when the instructor stands in front of it.
Inputs: Video feed of physical board
Outputs: Rectified images of the board surface.
Implementations:

- OpenCV Homography and Edge Detection for Perspective Correction. [1]
    - Definition: OpenCV provides image processing primitives (edges, corners) and homography transforms to unwarp a slanted/offset board back into a rectangle.
    - Open for research and commercial use
    - C++, Python, C
- ArUco fiducial markers for robust board localization.
    - Definition: ArUco are simple black-and-white square tags that the camera can recognize to infer the board's corners and pose.
    - Physical printed indicators to guarantee the correct board perspective.

Python + OpenCV

- **Grayscale Conversion:** cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
- **Noise Removal / Smoothing:**
    - Gaussian blur: cv2.GaussianBlur(image, (kernel_size, kernel_size), 0)
    - Median blur: cv2.medianBlur(image, kernel_size)
- **Thresholding / Binarization:**
    - Simple threshold: _, binary = cv2.threshold(gray, thresh_value, 255, cv2.THRESH_BINARY)
    - Adaptive threshold: cv2.adaptiveThreshold(...)
- **Edge Detection:**
    - Canny edge detection: edges = cv2.Canny(gray, lower_thresh, upper_thresh)

In ChalkStream, OpenCV will serve as the foundation of the image pre-processing pipeline. It will be used to detect and crop the board region through contour analysis, followed by perspective correction with image transforms so that angled views are resolved into a flat, rectangular image. Glare and lighting inconsistencies will be reduced using brightness equalization and inpainting, while gently enhancing stroke contrast for downstream OCR and diagram recognition. Together, these steps ensure that any processed video frame delivers a clean, normalized representation of the board, making recognition more accurate and reliable

4.1.2 Writing Extraction
Description: Filter out the background of the board and extract the writing. Aiming to categorize each pixel as written and unwritten areas. (0 and 1)
Inputs: output from Border adjustment (Rectified images of the board surface.)
Outputs: a picture that consists of only two colors (representing the written and unwritten area), that depicts the input picture's writing.
Implementations:

- OpenCV Homography and Edge Detection for Perspective Correction. [1]
    - Definition: OpenCV provides image processing primitives that allow us to filter out ink color from board color.

- Open for research and commercial use
- C++, Python, C

Adaptive Background Modeling is the proposed approach to reliably capture writings from the board while the presenter is obstructing the board. It gradually learns the board as the background while filtering out the moving professor. The learning rate is kept low enough so that newly written text is absorbed into the background and remains visible, but not so fast that the professor becomes part of the background. After background subtraction, apply morphological operations to remove noise, and optionally use thresholding to enhance the contrast of the text against the board. Once the clean background is reconstructed, feed those frames into an OCR engine (like Tesseract) to extract the writing. This pipeline ensures the professor is effectively removed while the board text remains sharp for recognition.

Initialize background model

- Use OpenCV's cv2.createBackgroundSubtractorMOG2() or cv2.createBackgroundSubtractorKNN().

Tune learning rate

- Set a low learning rate so that newly written text becomes part of the background.
- Ensure it's not too low, otherwise lighting changes will break the model.

Apply background subtraction

- For each frame, call fgmask = subtractor.apply(frame, learningRate=…).
- This produces a mask where the moving professor is foreground and the board + text remain background.

Clean up with morphological operations

- Use cv2.morphologyEx(fgmask, cv2.MORPH_OPEN, kernel) to remove noise.
- Optionally use cv2.MORPH_CLOSE to fill small gaps in the text.

Enhance text visibility

- Apply thresholding (cv2.threshold) to improve contrast between the board and text.
- Optionally apply smoothing to reduce leftover shadows.

Reconstruct the clean board

- Use the updated background model as the "board-only" frame.
- The professor is filtered out, while text stays visible.

Feed into OCR

- Pass the processed frame to an OCR engine (e.g., Tesseract) for text extraction.

4.2. Content Segmentation

Description: Separate the board into regions such as text lines, equation areas, and diagrams so that specialized recognizers can be applied.

Inputs: Preprocessed board images.

Outputs: Cropped regions labeled by type.
Implementations:

- OpenCV Homography and Edge Detection for Perspective Correction. [1]
    - Definition: OpenCV provides image processing primitives (edges, corners) and homography transforms to unwarp a slanted/offset board back into a rectangle.
    - Open for research and commercial use
    - C++, Python, C
- CRAFT text detector for text-line detection. [2]
    - Definition: CRAFT detects character regions and links them into lines, handling curved or irregular text.
    - Slower processing speeds, but
- YOLOv8 object detector trained for math/diagram regions.
    - Definition: A modern one-stage detector that can be fine-tuned to localize “equation” or “diagram” boxes in real time.

4.3. Handwriting OCR

Description: Convert handwritten words into editable, searchable text.
Inputs: Cropped text region images.
Outputs: Unicode text with bounding boxes and confidences.
Implementations:

- PaddleOCR
    - Definition: An open-source OCR system developed by Baidu, supporting printed and handwritten text recognition across multiple languages. It provides end-to-end pipelines for detection and recognition, pretrained models for handwriting, and integration with Python via pip packages.
    - Strengths: Lightweight, fast inference on CPU/GPU, wide language support, high accuracy on handwriting datasets, and active community support.
    - Relevance: Fits directly into ChalkStream’s vision pipeline for capturing whiteboard/chalkboard writing, ensuring real-time digitization.
- Tesseract OCR (fallback)
    - Definition: A classical open-source OCR engine developed by Google. While it is strongest for printed text, its legacy handwriting models provide a baseline comparison.
    - Relevance: Lightweight, easy to deploy, cross-platform, and useful as a baseline if PaddleOCR models fail or cannot load.

4.4. Math OCR

Description: Recognize equations and convert them into LaTeX/MathML.
Inputs: Cropped equation images.
Outputs: Structured LaTeX or MathML plus confidence.
Implementations:

- CROHME datasets and baselines.
    - Definition: A widely used benchmark of handwritten math expressions and evaluation tools.
- Mathpix API.
    - Definition: A commercial math OCR engine with state-of-the-practice accuracy for handwritten formulas.
- Im2LaTeX (image-to-markup).
    - Definition: A neural model that translates a formula image into LaTeX tokens using attention.

4.5. Diagram Vectorization
Description: Reconstruct graphs and illustrations as clean, resizable vector shapes.
Inputs: Cropped diagram regions.
Outputs: Vector primitives (lines, circles, arrows) in JSON or SVG.
Implementations:

- Potrace tracing algorithm.
    - Definition: Converts black-and-white bitmaps into smooth Bézier curves.
- Hough Transform for lines/circles.
    - Definition: A parametric voting method to detect simple geometric primitives in noisy drawings.
- Neural shape extraction from drawings.
    - Definition: A deep approach that detects and groups geometric elements (e.g., boxes, arrows) from sketches.

4.6. Speech-to-Text (ASR)
Description: Produce time-aligned lecture transcripts from the instructor's audio.
Inputs: Audio from classroom microphone.
Outputs: Time-stamped transcript segments.
Implementations:

- Whisper transformer ASR.
    - Definition: A multilingual speech recognizer trained on large, weakly supervised data, robust to accents/noise.
- Google Cloud Speech-to-Text.
    - Definition: A cloud service with streaming recognition and diarization options.
- Kaldi toolkit.
    - Definition: A modular, open-source ASR toolkit used for research and custom pipelines.

4.7. Multimodal Fusion and Structuring
Description: Synchronize board states with transcript segments to build a unified, searchable lecture timeline.
Inputs: Recognized text, equations, diagrams, and transcripts.
Outputs: Time-stamped, structured lecture representation (sections, events).
Implementations:

- Change-point detection for timeline segmentation.
    - Definition: Algorithms that find moments when the data distribution shifts (e.g., new board "state").
- Cross-modal alignment using embeddings (e.g., BERT).
    - Definition: Map text from transcripts and board into a shared vector space to align related content.
- Knowledge graph structuring.
    - Definition: Represent concepts and their relations (uses/proves/derives) as a graph for navigation and retrieval.

4.8. AI Services (Summarization, Translation, Q&A)
Description: Provide summaries, translations, and grounded answers linked to captured content.
Inputs: Structured lecture data and user queries/preferences.
Outputs: Summaries, translated text, and answers with citations to board spans.
Implementations:

- GPT-4/ChatGPT-class LLMs.

    - Definition: Large language models that generate explanations and summaries conditioned on retrieved context.
- MarianMT for neural translation.
    - Definition: An efficient NMT framework with many pretrained language pairs.
- Retrieval-Augmented Generation (RAG).
    - Definition: A pattern that retrieves source passages and conditions the generator to reduce hallucinations.

4.9. Storage and Delivery
Description: Persist artifacts securely and deliver real-time experiences to browsers.
Inputs: Board states, transcripts, vectors, and metadata.
Outputs: Secure storage; student app and instructor dashboard sessions.
Implementations:
- PostgreSQL
    - Definition: A relational DB suitable for real-time capabilities.
- S3/Azure Blob object storage.
    - Definition: Durable, scalable storage for raw video/audio and exported files.
- React + WebRTC front-end.
    - Definition: A modern web UI with peer-to-peer low-latency media and WebSocket/RTC data channels.

4.10 Host server
Description: Website for instructors to create classes and add students to enable access to live and stored lecture materials.
Inputs: Access to processed and saved lecture data
Outputs: User interface with accounts for students and professors, and access to system products.

5. How does ChalkStream relate to the degree program of each student?

ChalkStream involves software engineering techniques and skills that have been developed, acquired and practiced throughout our degree. It covers a wide range of topics and aspects of the software engineering program such as AI, machine learning, APIs, software design, software development, data analysis and storage. It follows the structured process of the Software Developmental Life Cycle where it involves planning, requirement analysis, design, development and deployment. It allows each student to apply the knowledge and experience acquired throughout their degree through courses, projects, assignments, labs, and internships into a project that addresses and solves a real life problem. It also allows students to encounter challenges and limitations that they have not faced in their school work, and provides them with the opportunity to face those challenges and use their problem solving skills and techniques they learned from the software engineering degree to tackle those problems. And finally designing a system that is useful, efficient and feasible.

6. A discussion of how the group, collectively, has the skills required to undertake the project.

All of the group members are working towards their last year of their software engineering program. Degree wise, the group members share similar skills regarding courses taken and year standings. Our group has strong programming skills in Python and Java which will be used extensively in AI training models, frontend and backend processing. Some of our group members have already taken the machine learning and the AI in engineering course, and some will be taking the courses in the winter term which will be highly useful as the implementation phase will be mostly in the winter. For computer vision and image processing component: expertise in computer vision techniques, including frame differencing,

background modeling, and contour analysis is critical for the preprocessing and board detection steps. Our group will be utilizing learning platforms such Kaggle and Coursera to deepen our understanding in machine learning, computer vision and image processing. As well as, referring to documentation, academic journals, research articles and library references. This approach allows us to stay updated with the latest methodologies, best practices, and cutting-edge techniques, ensuring that our project is informed by both practical applications and current research findings.

7. Method(s) you are going to use in solving the problem and how they relate to knowledge you acquired in your degree program to-date.

8. A proposed timetable for completion of the project including major intermediate milestones.

Start date: Oct 17. 2025
Finish date: April 5th 2026
20 week
Frame Processing:

- Prerequisite: None
- Number of people assigned: 3
- Estimated duration: 2 weeks
- Earliest Start: October 17, 2025
- Latest Start: October 31, 2025
- Earliest Finish: October 31, 2025
- Latest Finish: November 14, 2025

Content Segmentation

- Prerequisite: Frame Processing
- Number of people assigned: 3
- Estimated duration: 2 week
- Earliest Start: October 31, 2025
- Latest Start: November 14, 2025
- Earliest Finish: November 14, 2025
- Latest Finish: November 28, 2025

Handwriting OCR

- Prerequisite: Content Segmentation
- Number of people assigned: 1
- Estimated duration: 4 weeks
- Earliest Start: November 14, 2025
- Latest Start: November 28, 2025
- Earliest Finish: December 12, 2025
- Latest Finish: January 23, 2026

Math OCR

- Prerequisite: Content Segmentation
- Number of people assigned: 1
- Estimated duration: 3 week
- Earliest Start: November 14, 2025
- Latest Start: December 5, 2025
- Earliest Finish: December 5, 2025
- Latest Finish: January 23, 2026

Diagram Vectorization
- Prerequisite: Content Segmentation
- Number of people assigned: 1
- Estimated duration: 3 week
- Earliest Start: November 14, 2025
- Latest Start: December 5, 2025
- Earliest Finish: December 5, 2025
- Latest Finish: January 23, 2026

Speech-to-Text (ASR)
- Prerequisite: None
- Number of people assigned: 1
- Estimated duration: 1 week
- Earliest Start: November 14, 2025 (all members are busy on other tasks before then)
- Latest Start: week 9
- Earliest Finish: week 5
- Latest Finish: January 23, 2026

Multimodal Fusion and Structuring
- Prerequisite: OCR, Diagram Vectorization, Speech-to-Text
- Number of people assigned: 3
- Estimated duration: 3 week
- Earliest Start: December 12, 2025
- Latest Start: January 23, 2026
- Earliest Finish: January 30, 2026
- Latest Finish: February 13, 2026

AI Services (Summarization, Translation, Q&A)
- Prerequisite: Multimodal Fusion and Structuring
- Number of people assigned: 1
- Estimated duration: 1 week
- Earliest Start: January 30, 2026
- Latest Start: February 27, 2026
- Earliest Finish: February 6, 2026
- Latest Finish: March 6, 2026

Website and Storage

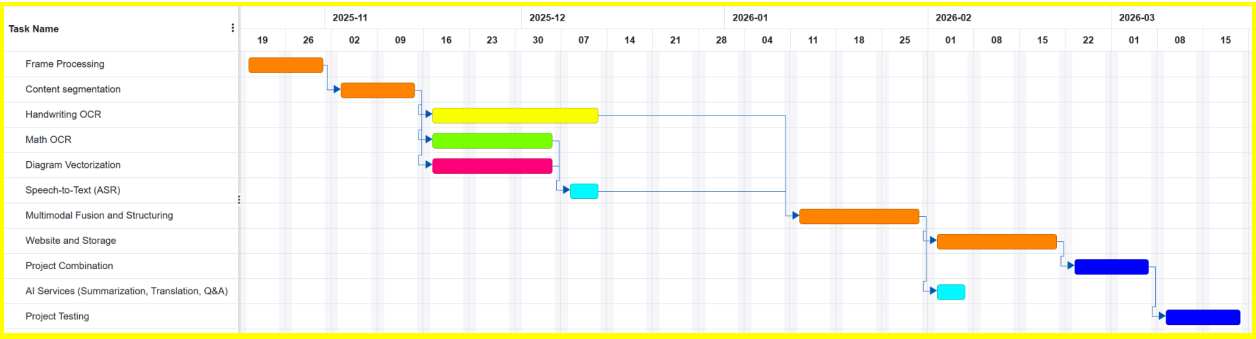
- Prerequisite: Multimodal Fusion and Structuring
- Number of people assigned: 3
- Estimated duration: 3 week
- Earliest Start: January 30, 2026
- Latest Start: February 13, 2026
- Earliest Finish: February 20, 2026
- Latest Finish: March 6, 2026

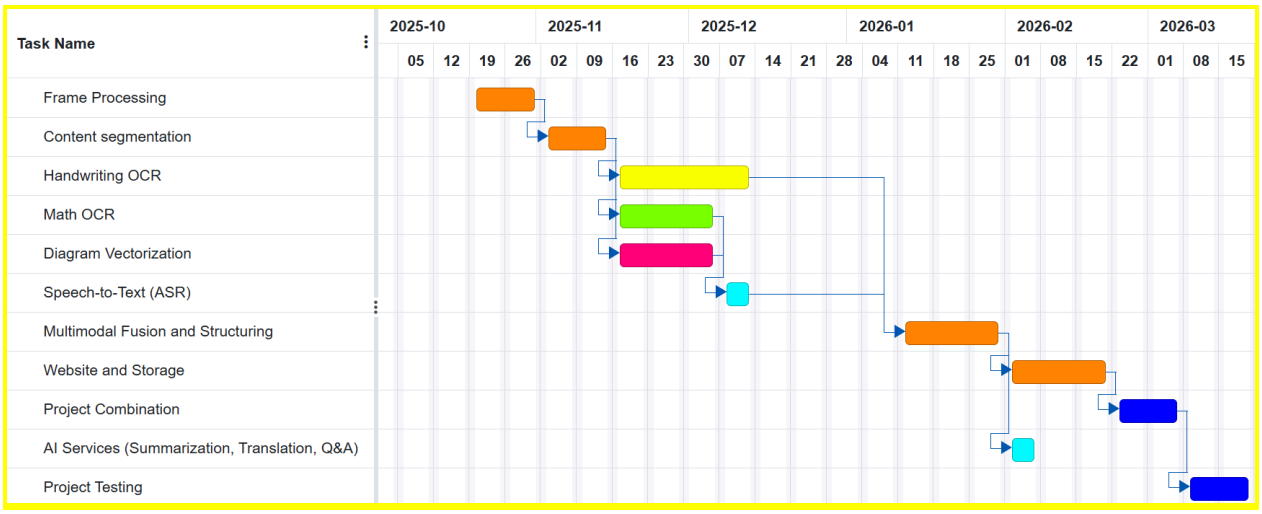
Project Combination

- Prerequisite: All of the above
- Number of people assigned: 3
- Estimated duration: 2 week
- Earliest Start: February 20, 2026
- Latest Start: March 6, 2026
- Earliest Finish: March 6, 2026
- Latest Finish: March 20, 2026

Project Testing

- Prerequisite: Project Combination
- Number of people assigned: 3
- Estimated duration: 2 week
- Earliest Start: March 6, 2026
- Latest Start: March 20, 2026
- Earliest Finish: March 20, 2026
- Latest Finish: April 3, 2026





9. Project risks and mitigation strategies.

9.1. Error in pre-processing

Description: There can be errors in the pre-processing process which result in poorly processed images. (Ex. board cropped unevenly, script fails to capture all written portion) This cause the final output to be unstable

Risk likelyhood: Very High

Risk impact: Very low

Proposed Solutions:

- Test run requirement:
  - Professors are required to conduct a test run and adjust the sensitivity of the image crop, ensuring the board is correctly cropped.
- Frame Comparison:
  - Each processed frame will be compared with adjacent frames for crop alignment and writing alignment. Areas in frames that have a lot of variation over adjacent frames will be ignored

9.2. Error in AI process

Description: AI errors can occur which result in wrongly received drawing/writings.

Risk Likelihood: medium

Risk Impact: low

Proposed solution:

- Manual Selection
  - Students may manually select what type the output is (Diagrams/words/math equation) and the correct AI will transcribe the output
- Original option
  - Students may choose to use the pre-processed image as an option.

9.3. External API down

Description: External API may not be always active. The project may not work properly if the relied APIs are down.
Risk Likelihood: low
Risk Impact: Medium
Proposed Solution:

- Feature adaption
    - All procedures of the running project should be prepared to handle the absence of any API features. Quality is expected to be reduced, but all procedures should remain functional.
- API check
    - The system check the availability of all API, allowing the program to adapt beforehand
    - Features built on user UX will be greyed out if the API are not available

9.4. Server shutdown due to fatal error

Description: Servers shutdown unexpectedly due to fatal errors. Halting class
Risk Likelihood: very-low
Risk Impact: High
Proposed Solution:

- Emergency Save
    - The ongoing lecture will save the notes before the server fully shuts down. The program will have the ability to resume the lecture from saved files, allowing the lecture to still be saved in one file.

10. Required special components and facilities that you require.

11. References

[1] J. Howse and J. Minichino, Learning OpenCV 4 Computer Vision with Python 3, 3rd ed. Packt Publishing, 2020. (Available through Carleton Omni)

[2] Baek, Y., Lee, B., Han, D., Yun, S., & Lee, H. (2019). Character Region Awareness for Text Detection (CRAFT). In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2019), 9365–9374.(Available online)